
Module 04 — HTTP Clients and Resilience

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — Review Service Client (Easy)	2
Exercise 2 — Retry and Fallback (Medium)	3
Exercise 3 — Protecting a Partner from Overload (Hard)	3
Bonus Challenge (Optional)	4

How to Work

1. The starter code is in `modules/04-http-clients-resilience/exercise/`. Find the `TODO` comments.
2. Every exercise already has tests, and they stay **red** until you solve it. A single WireMock server fakes the partner services (`PartnerServerTest`).
3. Run the tests:

```
./mvnw -Pexercises -pl modules/04-http-clients-resilience/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/04-http-clients-resilience/solution/`.

★ Tip

To run the tests of a single exercise: `-Dtest=Exercise1Test -Dsurefire.failIfNoSpecifiedTests=false`

Exercise 1 — Review Service Client (Easy)

Goal: define an HTTP client with nothing but an interface.

The partner's review service offers two endpoints: `GET /reviews/{isbn}` (the list of reviews) and `POST /reviews/{isbn}` (a new review as the JSON body).

Tasks (package `exercise1`):

- `TODO 1a` — all methods of `ReviewApi` live under the path `/reviews`.
- `TODO 1b` — `forBook(isbn)`: `GET /reviews/{isbn}`.
- `TODO 1c` — `add(isbn, review)`: `POST /reviews/{isbn}`, the review goes as the JSON body.
- `TODO 1d` — register `ReviewApi` as an HTTP service client in the group `reviews`. Its address comes from `spring.http.serviceclient.reviews.base-url`.

Hints:

- Lesson section 3.4: `@HttpExchange`, `@GetExchange`, `@PostExchange`, `@PathVariable`, `@RequestBody`.
- `@ImportHttpServices(group = ..., types = ...)`.

Acceptance criteria: both tests in `Exercise1Test` pass.

Estimated time: 20 minutes

Exercise 2 — Retry and Fallback (Medium)

Goal: retry transient failures, and give a fallback answer for persistent ones without crashing.

The stock service sometimes answers 502 or 503. When the stock is unknown, the order page must show “no stock information”.

Tasks:

- TODO 2a — ResilienceConfiguration: switch on Spring Framework 7’s resilience features. Exercise 3 needs this too.
- TODO 2b — StockClient.available: retry server errors (5xx) only, at most 2 times, 50 ms apart.
- TODO 2c — StockFacade.availableOrUnknown: if the failure persists after the retries, return UNKNOWN instead of throwing.

Hints:

- Lesson section 3.6: @EnableResilientMethods, @Retryable(includes = ..., maxRetries = ..., delay = ...).
- The exception RestClient throws for 5xx: HttpServerErrorException. The parent of all client errors: RestClientException.
- Why do the retried method (StockClient) and the fallback (StockFacade) live in separate beans?

Acceptance criteria: all 3 tests in Exercise2Test pass. The tests also check the number of requests (1 + 2 retries, a single request for a 404).

Estimated time: 30 minutes

Exercise 3 — Protecting a Partner from Overload (Hard)

Goal: use both policies of the concurrency limit (BLOCK and REJECT), and handle a rejected call gracefully.

The shipping partner allows at most **1 express** and **2 standard** quote requests at the same time. When the express slot is busy, the customer should immediately see a standard price instead of waiting.

Tasks (package exercise3):

- TODO 3a — QuoteService.expressQuote: at most 1 call at a time. A second caller is **refused** without waiting.
- TODO 3b — QuoteService.standardQuote: at most 2 calls at a time; further callers wait.
- TODO 3c — QuoteFacade.expressOrStandard: when the express call is refused, fetch a standard quote and return "standard".

Hints:

- Lesson section 3.7: `@ConcurrencyLimit(limit = 1, policy = ConcurrencyLimit.ThrottlePolicy.REJECT)`.
- A refused call throws `org.springframework.resilience.InvocationRejectedException`.

Acceptance criteria: both tests in `Exercise3Test` pass.

Estimated time: 40 minutes

Bonus Challenge (Optional)

Add a read timeout to `StockClient`: set 500 ms for this client only, through the `RestClient.Builder` instead of `spring.http.serviceclient`. Write a test with `WireMock`'s `withFixedDelay(2000)` showing that the timeout turns into an UNKNOWN answer.